arXiv:2311.11501v1 [cs.LG] 20 Nov 2023

# MULTILORA: DEMOCRATIZING LORA FOR BETTER MULTI-TASK LEARNING

**Yiming Wang, Yu Lin, Xiaodong Zeng, Guannan Zhang**
Ant Group
Shanghai, China

**ABSTRACT**

LoRA achieves remarkable resource efficiency and comparable performance when adapting LLMs for specific tasks. Since ChatGPT demonstrated superior performance on various tasks, there has been a growing desire to adapt one model for all tasks. However, the explicit low-rank of LoRA limits the adaptation performance in complex multi-task scenarios. LoRA is dominated by a small number of top singular vectors while fine-tuning decomposes into a set of less important unitary transforms. In this paper, we propose MultiLoRA for better multi-task adaptation by reducing the dominance of top singular vectors observed in LoRA. MultiLoRA scales LoRA modules horizontally and change parameter initialization of adaptation matrices to reduce parameter dependency, thus yields more balanced unitary subspaces. We unprecedentedly construct specialized training data by mixing datasets of instruction follow, natural language understanding, world knowledge, to cover semantically and syntactically different samples. With only 2.5% of additional parameters, MultiLoRA outperforms single LoRA counterparts and fine-tuning on multiple benchmarks and model scales. Further investigation into weight update matrices of MultiLoRA exhibits reduced dependency on top singular vectors and more democratic unitary transform contributions[1].

## 1 Introduction

In recent years, Large Language Models (LLMs) have manifested unprecedentedly superior performance in various natural language processing tasks[1, 2, 3, 4]. As model scales up, high-level multi-task capabilities[5] emerges from LLMs. Capabilities such as real-world knowledge, logic reasoning and arithmetic skills from one LLM proves the feasibility of "one model for all tasks". However, scaling up LLMs by adding billions of parameters not only bring emergent abilities and grokking, but also dramatically increase training and down-stream adaptation costs. For instance, parameter counts of LLaMA[4] series range from 7 billion to 65 billion, and GPT-3[2] contains up to 175 billion parameters. Full parameter fine-tuning these models for down-stream adaptation yields huge amount of memory footprint and thus requires prohibitively expensive hardwares.

To address the issue of hardware requirements for LLM adaptation, a solution called Parameter Efficient Fine-Tuning (PEFT) has been proposed. PEFT methods reduce VRAM usage of cached optimizer states[6] by only optimizing a fraction of model parameters while keeping the rest frozen. Various PEFT methods, such as adapter[7], p-tuning[8], $IA^3$[9] and LoRA[6], have been suggested. Compared to other PEFT methods, LoRA possesses the advantages of: 1) high modularity for distribution, 2) mergeable weights for zero inference overhead. While LoRA has proven successful in single-task adaptation, its performance in more intricate multi-task settings of generative AI remains unexplored. Thus, a crucial question lingers: Can LoRA effectively adapts LLMs to complex multi-task scenarios as full parameter fine-tuning does?

Works on applying PEFT methods on multi-task learning scenarios are in literature, albeit with certain limitations[10, 11, 12, 13]. These proposed methods manage to improve multi-task benchmark performances with task information sharing or activation routing[13, 10, 11]. However, these dedicated modules add unaffordable overhead to transformer inference, which hinders their industrial application[6, 14]. Another limitation is that the prior works focused on Natural

[1]Our code is coming to GitHub soon.

Language Understanding (NLU), which may not be suitable for current generative LLMs. A mixture of NLU tasks are commonly used[10, 11] despite that data samples of these tasks do not present much semantical or syntactical difference among them. More benchmarks on tasks of interest of generative LLMs, like instruction following, logic reasoning should be taken into consideration.

Therefore, the research goal of this paper is to adapt LoRA for better multi-task learning while maintaining modularity and zero inference overhead of LoRA. We firstly reveal the fundamental difference between LoRA and full parameter fine-tuning with the help of Singular Value Decomposition (SVD, Section 3.2). We found dominance of top singular vectors in LoRA while fine-tuning is more democratic, as the residual weight decomposes into larger sets of unitary transforms of smaller importance. In order to mitigate the observed dominance, we propose to horizontally scale LoRAmodules (Section 3.3). MultiLoRA horizontally scales lora modules to reduce parameter dependency. MultiLoRA divides LoRA along the rank, add learnable scaling factor and change the parameter initialization to enhance expressiveness of lora modules. Compared to conventional LoRA, MultiLoRA produces more democratic weight update matrices as those of full parameter fine-tuning.

To better demonstrate the effectiveness of MultiLoRA, we constructed a comprehensive dataset composed of various tasks relevant to generative LLMs. A series of datasets from different domains are selected including instruction following[15], world knowledge[16], arithmetic reasoning[17] and NLU[18]. Both context and target of samples in aforementioned datasets exhibit strong semantical and syntactical differences, thus augmenting adaptation difficulty. With our multi-task datasets, we conducted extensive empirical experiments on LLaMA ranging from 7B to 65B. On benchmarks of MMLU[16] and SuperGLUE[18], we found MultiLoRA consistently outperforms LoRA even under smaller parameter budget and can perform on-par with full-parameter fine-tuning. We further dive into obtained weight update matrices with SVD. Side by side comparison to full parameter fine-tuning suggests that MultiLoRA exhibit a higher degree of subspace overlapping and more similar singular value distribution, indicating successful democratization of unitary transforms contribution.

Therefore, our main contributions can be summarized as follows:

- We find dominance of unitary transforms in weight update matrices of LoRA, while fine-tuning produces more democratic contribution distribution.
- We propose MultiLoRA to mitigate dominance seen in LoRAand democratize contributions of its unitary transforms.
- We propose a multi-task learning scheme based on mixture of tasks of interest of generative LLMs, to cover semantically and syntactically different samples. Our proposed MultiLoRA exhibits stronger consistency than LoRA and can outperform full parameter fine-tuning on various tasks and model scales.

## 2 Related Work

### 2.1 PEFT

PEFT methods lowers hardware requirement of model fine-tuning by significantly reducing trainable parameters and consequently optimizer states cached in VRAM. By exploiting the local optimum of a pretrained model, a much smaller solution space brought by reduce trainable parameters helps PEFT methods achieve comparable tuning performance[19, 20]. PEFT can be classified into two categories: 1) reparameterization-based methods[21, 22] that retrain a portion of the parameters and 2) addition-based methods that train additional parameters[6, 23, 7]. Recent works in PEFT focus on resource efficiency[7, 9, 6, 23]. LoRA[6] fits incremental weights by decomposing them into low-rank matrices. $(IA)^3$ tunes hidden states with learned multipliers. AdaLoRA[23] adds importance-aware pruning mechanisms to further improve resource efficiency. There're also work focusing on ensemble learning with adapters. AdaMix[13] and UniPELT[24] integrate existing PEFT methods into a unified framework to boost adaptation performance.

### 2.2 Multi-Task Learning with PEFT

In multi-task learning with PEFT, adapter is utilized for code summarization across different programming languages[12]. HyperFormer[10] assigns task-related weights to adapter[7] activations using shared hypernets across layers and tasks. Multitask Prompt Tuning[11] extends prompt tuning by firstly distilling from source prompts adapted for various tasks and further finetunes with low rank updates. While these methods have shown effectiveness, the additional weights cannot be seamlessly integrated into the base model, resulting in inevitable inference latency that is impractical for LLM serving[6, 14]. Moreover, the emphasis in the multi-task setting has predominantly been on NLU tasks, disregarding the tasks that are of interest to generative LLMs.

# 3 Method

## 3.1 Background

Before formal explanation on design choices of MultiLoRA, a few notations are proposed base on LLaMA and LoRA.

### 3.1.1 LLaMA

LLaMA model consists of $L$ stacked decoder layers, where each block contains two submodules: a multi-head attention (MHA) and a fully connected FFN. Given the input sequence $\mathbf{x} \in \mathbb{R}^{n \times d}$, MHA performs the attention function in parallel h heads:

$$\text{head}_i = \text{Softmax}(\frac{\mathbf{x}W_i^{q_proj}(\mathbf{x}W_i^{k_proj})^\top}{\sqrt{d_n}})\mathbf{x}W_i^{v_proj}, \text{MHA}(\mathbf{x}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_n)W^{o_proj}, \quad (1)$$

where $W_i^{q_proj}, W_i^{k_proj}, W_i^{v_proj} \in \mathbb{R}^{d \times d_h}$ are query, key and value projections of $i^{th}$ attention head and $W^{o_proj} \in \mathbb{R}^{d \times d}$ is the output projection to aggregate multi-head outputs. $d_h$ is typically set to $d$/h. The other important module is a MLP which consists of three linear transformations, namely *up_proj, down_proj, gate_proj* with a SwiGLU activation in between:

$$\text{MLP}(\mathbf{x}) = \text{SwiGLU}(\mathbf{x}W^{up_proj}(\mathbf{x}W^{gate_proj}))W^{down_proj}, \quad (2)$$

where $W^{up_proj}, W^{gate_proj} \in \mathbb{R}^{d \times d_{mid}}, d_{mid} > d$ and $W^{down_proj} \in \mathbb{R}^{d_{mid} \times d}$. Layer normalization is applied before and after the attention module.[4]

### 3.1.2 Low-Rank Adaptation

Given target module with weight $W \in \mathbb{R}^{d \times k}$, LoRA inserts two sequential low rank matrices to fit the residual weights for adaptation. The forward computation of adapted module writes as follow:

$$\mathbf{y}' = \mathbf{y} + \Delta\mathbf{y} = W\mathbf{x} + BA\mathbf{x}, \quad (3)$$

where $A \in \mathbb{R}^{d \times r}, B \in \mathbb{R}^{r \times k}$ with $r \ll \min(d, k)$. Either $A$ or $B$ is initialized with zeroes and the other is initialized with Kaiming Uniform[25] to force $\Delta\mathbf{y} = 0$ at the very beginning. Analysis on weight update matrices suggest that LoRA work by enhancing existing feature transforms in original model weight[6].

## 3.2 Difference between LoRA and fine-tuning

Although LoRA achieves comparable performances to fine-tuning on many benchmarks, it is essential to understand the underlying differences between the two approaches. To shed light on this question, We train LLaMA-7B on Alpaca and MMLU using both methods[2] to get weight update matrices $\Delta W$ and conduct an analysis of the weight update matrices using SVD.

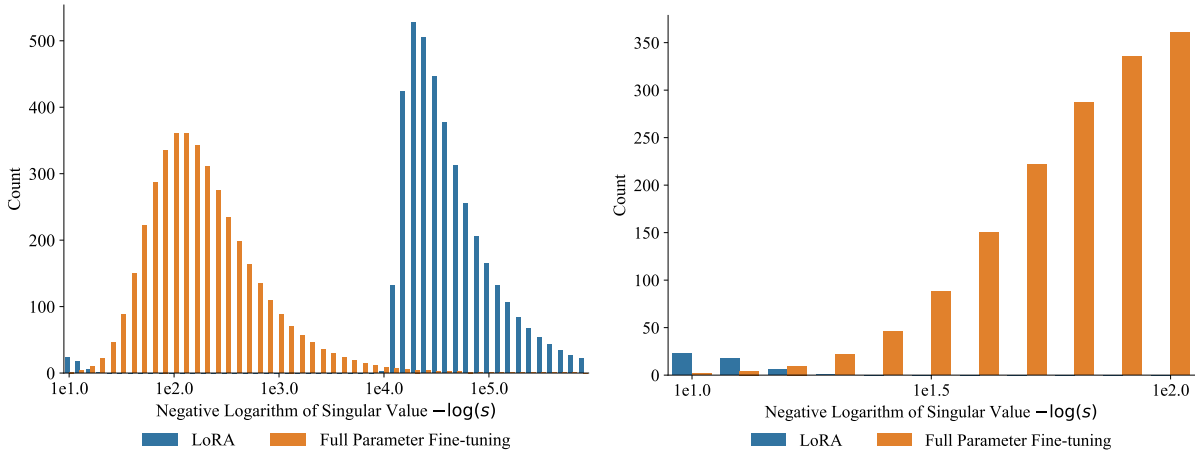
Figure 1 illustrates singular value distribution of $\Delta W^{FT}$ and $\Delta W^{LoRA}$. For better visualization, we plot the negative logarithms of the singular values $(-\log(s))$ . The empirical distribution of fine-tuning exhibits a bell-shaped curve while the distribution for LoRA falls at both ends of the spectrum. The extreme bimodal distribution of LoRA arises from the constraint that $Rank(\Delta W^{LoRA})$ should not exceed $r$, resulting in at least $(k - r)$ singular values being zero.

Interestingly, we also noticed an inverse trend in the counts of top singular values. In LoRA, the count increased with the magnitude of the singular values, while fine-tuning exhibited the opposite behavior. This suggests that LoRA predominantly relies on a small group of singular vectors, whereas fine-tuning distributes importance more evenly among singular vectors. Such phenomenon can also be observed on LoRA trained on other datasets or publicly available LoRA weights, indicating observed dominance arises from the structural design of LoRA (refer to Appendix B for more examples).

Based on these findings, we can infer that the dominance observed in LoRA may limit its adaptation performance, particularly in complex multi-task scenarios that require enhancement of multiple distinct feature transforms. $\Delta W$ of full parameter fine-tuning decomposes into a larger set (more specifically equals rank of original weight matrix) of unitary transforms. In contrast, LoRA's explicit rank limitation restricts it to decompose into a smaller number ($r$) of unitary transforms. As a result, the expressiveness of LoRA may be constrained compared to full parameter fine-tuning.

## 3.3 Scaling LoRA to Democratize Unitary Transform Contribution

[2]LoRA hyperparameters set to $r = 64$ and $\alpha = 64$




(a) (b)

Figure 1: Top singular value distribution of weight update matrix of $\Delta W_{v_proj}$. (a) Complete view of the histogram. (b) Close-up view on top singular values. Both histograms are plotted based on on the negative logarithms of the singular values $-\log(s)$, where left end of horizontal axis represents larger singular values. Bell-shape curved of full parameter fine-tuning indicates a democratic composition of a large number of relatively less important unitary transforms. On the hand, LoRA heavily relies on a small group of important unitary transforms, which could hurt complex multi-task adaptation.

In Section 3.2, we observe a small group of top singular vectors dominate weight update matrices of LoRA $\Delta W^{LoRA}$ while top singular vectors contribute more evenly to $\Delta W^{FT}$. In order to match fine-tuning in complex task adaptation, we propose MultiLoRA aiming at producing less polarized weight update matrices $\Delta W$. MultiLoRA inserts multiple parallel LoRAs to reduce parameter sharing, changes parameter initialization to enable larger optimization search space and implement starting point initialization with a learnable parameter. Figure 2 shows the overview of MultiLoRA. There're 3 major difference to original LoRA: horizontal scaling of LoRA modules, scaling factors and parameter initialization.

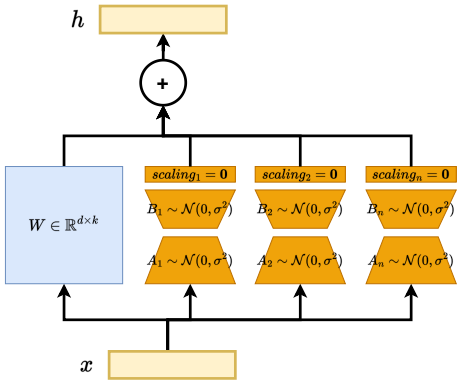



Figure 2: Overview of MultiLoRA. Multiple parallel LoRA modules are used to adapt target weight matrix. Parameter initialization and zero-initialized scaling factor are introduced to democratize residual weight updates.

### 3.3.1 Scaling LoRA Horizontally

Given that LoRA performs closely despite scaling up the rank $r$, our key strategy of depolarization is parallelism. Through parallelism, incremental activation $\Delta\mathbf{y}$ is further decomposed into a series of independent variables, which allows for more degrees of freedom during optimization. Bear in mind that under the same parameter budget, decomposing one large LoRA module into multiple small LoRAs cannot augment rank of $\Delta W$ as $rank(AB) \leq \min(rank(A), rank(B))$ and $rank(A+B) \leq rank(A) + rank(B)$. Corresponding weight matrices are noted as $\{A_i \in \mathbb{R}^{r\times k}\}_{i\in[1,n]}, \{B_i \in \mathbb{R}^{d\times r}\}_{i\in[1,n]}$. Thus, the forward computation of MultiLoRA writes as:

$$\Delta\mathbf{y} = \sum_{i=1}^{n} scaling_i B_i A_i \mathbf{x}, \tag{4}$$

Comparing with the forward of LoRA, MultiLoRA differs in that less parameter dependency brought to $\{B_i\}$. Parallelism has no effect on $A$ as intermediate $m_i = A_i\mathbf{x}$ is equivalent to reshape the result of $[A_0^\top, \ldots, A_n^\top]^\top\mathbf{x}$. But here comes the major difference.

#### 3.3.2 Parameter Initialization

Scaling LoRA horizontally allows for independent feature transform especially the up-projection of $\{B_i\}$. To further push the expressiveness of $\{B_i\}$, we change its parameter initialization to Kaiming-Uniform [25] instead of all zeroes and consequently introduce a learnable scaling factor to implement the starting point initialization.

The zero initialization seen in $B$ of LoRA aims to keep activation unchanged before training. Such practice, we term as *Starting Point Initialization*, is commonly seen in PEFT methods but may be implemented differently. With starting point initialization, tuning a pretrained LLM essentially becomes optimizing in a much smaller parameter space around the local optimum of pretrained models. However, zero initialization is a double-edged sword. It is also infamous for introducing redundancy and breaks asymmetry[26, 25], yielding limited expressiveness of networks despite faster convergence speed during adaptation.

To take advantage of the starting point initialization and mitigate the drawbacks of zero initialization, Multi-LoRA changes initialization of $\{B_i\}$ to Kaiming-Uniform and implements stating point initialization with zero-initialized learnable scaling factors $scaling_i \in \mathbb{R}^k$. Kaiming-Uniform has been shown to improve the generalization performance of neural networks and is the default parameter initialization method in PyTorch[27] implementation.

## 4 Experiments

In this section, we evaluate our proposed method from three aspects, namely memory profile, throughput and downstream performance. All our experiments are conducted with LLaMA series[4], ranging from 7B to 65B.

### 4.1 Experiment Setups

| Model Size | Method | MMLU | Boolq | MultiRC | RTE | WIC |
|---|---|---|---|---|---|---|
| 7B | Zero-Shot | 35.1 | 66.5 | 42.3 | 57.0 | 49.4 |
| | FT | **45.3** | 87.6 | **84.5** | **87.0** | **71.2** |
| | $\text{LoRA}_{r=96}$ | 44.7 | 86.0 | 81.7 | 86.6 | 67.6 |
| | $\text{MultiLoRA}^{n=3}_{r=32}$(Ours) | 45.1 | **88.7** | 83.8 | 85.6 | 70.2 |
| 13B | Zero-Shot | 46.9 | 65.0 | 43.4 | 60.6 | 49.5 |
| | FT | **51.3** | 87.1 | 85.7 | 90.8 | 74.3 |
| | $\text{LoRA}_{r=96}$ | 51.0 | **87.3** | **86.1** | **91.7** | 69.9 |
| | $\text{MultiLoRA}^{n=3}_{r=32}$(Ours) | 51.3 | 86.7 | 84.7 | 91.4 | **75.4** |
| 30B | Zero-Shot | 57.8 | 74.6 | 46.9 | 53.4 | 50.0 |
| | FT | **59.2** | 89.3 | 87.9 | 92.8 | 74.0 |
| | $\text{LoRA}_{r=96}$ | 58.8 | **89.7** | 87.0 | 91.0 | 74.1 |
| | $\text{MultiLoRA}^{n=3}_{r=32}$(Ours) | 59.1 | 89.5 | **88.1** | **93.1** | 74.1 |
| 65B | Zero-Shot | 63.5 | 73.6 | 48.3 | 59.6 | 51.3 |
| | FT | **64.6** | **91.6** | 90.1 | **93.9** | 75.4 |
| | $\text{LoRA}_{r=96}$ | 64.2 | 91.4 | 90.0 | 93.1 | 74.5 |
| | $\text{MultiLoRA}^{n=3}_{r=32}$(Ours) | 63.3 | 91.0 | **90.2** | 93.5 | **74.7** |

Table 1: Main results on MMLU and SuperGLUE using LLaMA of all scales trained in conventional single dataset setup. MMLU is tested with 5-shot prompts and SuperGLUE are tested with zero-shot. MultiLoRA, LoRA and full parameter fine-tuningproduces similar results on single dataset setup.

#### 4.1.1 Training Data

To evaluate on tasks of interest of generative LLMs, we build multi-task datasets encompassing Alpaca[15] for instruction following, MMLU[16] for world knowledge, GSM8K[17] for arithmetic reasoning and SuperGLUE[18] for NLU. Therefore, our mixture of tasks covers semantically and structurally different samples. In terms of source and target sequence length, samples from MMLU and SuperGLUE consist of single-choice questions with very short target lengths, typically one token. On the other hand, Alpaca and GSM8k contain longer target sequences. From the aspect of task semantic, the subjects covered by each dataset differ. MMLU encompasses real-world knowledge across various domains such as humanities, STEM, and social sciences, offering different levels of difficulty. In contrast, Alpaca focuses primarily on aligning model output with human preferences. GSM8k train the models to generate logical and step-by-step responses to questions.

To ensure consistency in evaluation, we follow QLoRA[28] and MeZO[29] to verbalize samples of MMLU and SuperGLUE, respectively. This verbalization process helps standardize input data across tasks, enabling fair comparisons. During training, we introduce random shuffling to enhance the learning process and prevent any bias that may arise from the ordering of the samples.

#### 4.1.2 Baselines

We use models from LLaMA[4] series as the base model. In our comparative analysis, we consider two baselines: full parameter fine-tuning (referred to as **FT**) and single **LoRA** (referred to as LoRA). To establish a strong single LoRA baseline, we incorporate LoRA modules alongside all linear layers of LLaMA. Specifically, we insert LoRA modules in *q_proj, k_proj, v_proj, o_proj, up_proj, down_proj, gate_proj* modules in LLaMA. The more layers that are adapted by LoRA, the better down-stream task performances will be[28, 6].

For Boolq, MultiRC, RTE and WIC, we report zero-shot performances and we report 5-shot results for MMLU. Instead of reporting the individual best task scores, we report the score of each task when the best average score is achieved to emphasize multi-task capability. The hyperparameter settings employed in our experiments are detailed in Appendix A. All experiments are conducted using 8 A100 80G GPUs . Python library PEFT[30] is used to help implement MultiLoRA and LoRA. We use Deepspeed ZeRO-3[31] for distributed training and offload optimizer states and model parameters for larger training throughput.

### 4.2 Evaluation Results

| Model Size | Method | # Params | MMLU | Boolq | MultiRC | RTE | WIC | AVG. |
|---|---|---|---|---|---|---|---|---|
| 7B | FT | 100% | 49.5 | 88.4 | 87.2 | 85.2 | **74.0** | 76.9 |
| | LoRA$_{r=96}$ | 3.6% | 47.7 | 88.2 | 85.4 | 83.4 | 71.6 | 75.2 |
| | LoRA$_{r=160}$ | 5.9% | 50.2 | 87.7 | 85.3 | 83.3 | 70.1 | 75.3 |
| | MultiLoRA$^{n=3}_{r=32}$(Ours) | 3.6% | 51.2 | 87.8 | 88.7 | **89.7** | 70.8 | 77.6 |
| | MultiLoRA$^{n=5}_{r=32}$(Ours) | 6.0% | **51.4** | **88.5** | **89.4** | 89.4 | 71.4 | **78.0** |
| 13B | FT | 100% | 51.4 | 89.2 | 89.3 | **91.3** | **75.1** | 79.2 |
| | LoRA$_{r=96}$ | 2.9% | 49.7 | **89.7** | 88.5 | 87.0 | 71.5 | 77.2 |
| | LoRA$_{r=160}$ | 4.8% | 50.4 | 89.4 | 88.4 | 87.6 | 72.1 | 77.5 |
| | MultiLoRA$^{n=3}_{r=32}$(Ours) | 2.9% | 52.6 | 89.4 | **89.9** | 86.9 | 74.1 | 78.5 |
| | MultiLoRA$^{n=5}_{r=32}$(Ours) | 4.8% | **52.9** | 89.3 | 89.5 | 90.3 | 74.3 | **79.4** |
| 30B | FT | 100% | 57.5 | 90.5 | **91.0** | 91.7 | **75.9** | **81.3** |
| | LoRA$_{r=96}$ | 2.2% | 57.1 | 90.2 | 90.5 | 90.5 | 74.0 | 80.4 |
| | LoRA$_{r=160}$ | 3.7% | 56.8 | 90.8 | 90.1 | 89.9 | 73.8 | 80.2 |
| | MultiLoRA$^{n=3}_{r=32}$(Ours) | 2.3% | **58.4** | 90.6 | 90.5 | 91.5 | 74.9 | 81.1 |
| | MultiLoRA$^{n=5}_{r=32}$(Ours) | 3.8% | 58.0 | **91.7** | 90.6 | **91.9** | 75.2 | 81.2 |
| 65B | FT | 100% | **66.4** | 91.7 | **91.3** | **93.9** | 76.5 | **83.9** |
| | LoRA$_{r=96}$ | 1.8% | 65.9 | 91.3 | 90.8 | 92.4 | 75.1 | 83.1 |
| | LoRA$_{r=160}$ | 3.1% | 65.8 | 90.9 | 90.4 | 93.6 | 75.5 | 83.2 |
| | MultiLoRA$^{n=3}_{r=32}$(Ours) | 1.8% | 65.9 | 91.5 | 90.5 | 93.8 | 76.2 | 83.5 |
| | MultiLoRA$^{n=5}_{r=32}$(Ours) | 3.1% | 66.3 | **91.8** | 90.1 | 93.3 | **76.6** | 83.6 |

Table 2: Evaluation results on MMLU and SuperGLUE using LLaMA of all scales trained on our mixture. We report score of each task when the best average score is achieved throughout training. MMLU is tested with 5-shot prompts and SuperGLUE are tested with zero-shot. MultiLoRA produces better and more consistent results compared to LoRA.

Benchmark performances trained on mixed data are listed in Table 2. Comparing these results to those obtained from single datasets (listed in Table 1), training on mixed datasets generally leads to benefits across all tasks and model scales, resulting in improved benchmark scores to varying extents. Based on these findings, we draw the following conclusions:.

**MultiLoRA consistently outperforms LoRA and achieves better results than full parameter fine-tuning on smaller models.** Across all benchmarks and model scales, MultiLoRA demonstrates stronger data fitting capabilities and outperforms the LoRA counterpart with the same parameter budget by a notable margin. For instance, MultiLoRA improves upon LoRA 's performance by 3.5% on MMLU for LLaMA-7B and by 5.9% on RTE for the same model. On average, MultiLoRA surpasses LoRA in terms of the evaluated tasks' average score by 2.8%. Notably, MultiLoRA even outperforms full parameter fine-tuning on smaller models (7B and 13B), only slightly falling behind on larger scales. Specifically, MultiLoRA achieves an average score improvement of 1.1% compared to full parameter

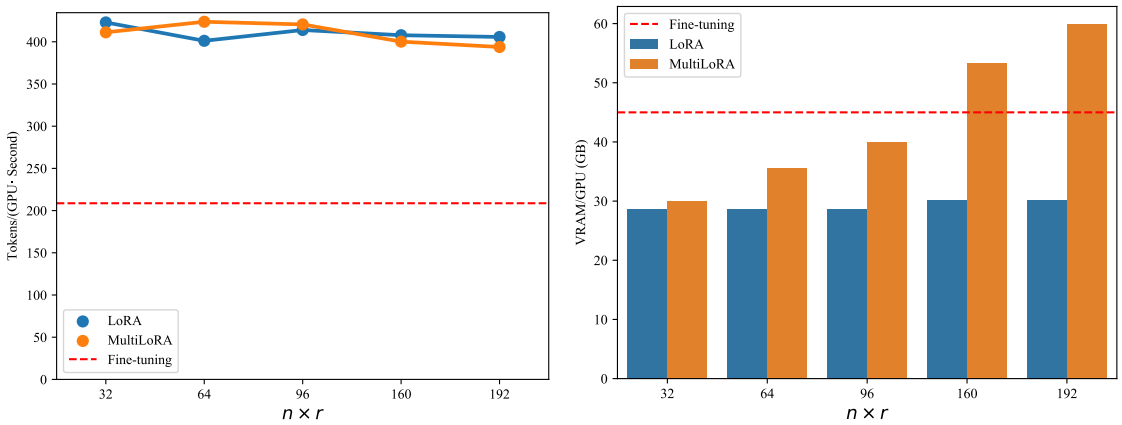
fine-tuning on LLaMA-7B, and a slight 0.3% decrease on LLaMA-65B. These significant improvements highlight MultiLoRA 's superior capability in complex multi-task adaptation.

**MultiLoRA exhibits small performance fluctuations comparable to full parameter fine-tuning in complex multi-task learning scenarios.** On smaller models, LoRA tends to show performance variability, with more frequent fluctuations between different tasks. For example, on LLaMA-7B, compared to full parameter fine-tuning , MultiRC, RTE, and WIC scores exhibit fluctuations of over 3% in LoRA . In contrast, both full parameter fine-tuning and MultiLoRA yield consistent individual task scores. The observed fluctuations in LoRA can be attributed to the dominance of top singular vectors, as noted in Section 3.2, where a small number of unitary transforms carry significant importance.

**In the single dataset setting, MultiLoRA performs similarity to full parameter fine-tuning and LoRA.** Table 1 shows evaluation results of models trained on single dataset. Across the 5 tested tasks and 4 scales, full parameter fine-tuning performs the best on 9 combinations, while MultiLoRA and LoRA perform the best in 7 and 5 combinations, respectively (MultiLoRA performs equally to LoRA on WIC for LLaMA-30B). Based on these observations, we cannot definitively declare one approach as superior. However, in the multi-task setting, MultiLoRA and full parameter fine-tuning demonstrate better performances compared to LoRA.

### 4.3 Resources & Throughput Analysis

Training throughput, VRAM usage and inference latency are crucial for generative LLMs. In this section, In this section, we thoroughly examine the resource usage and throughput of MultiLoRA as we scale up the number of parallel LoRA modules $n$. We primarily focus on VRAM usage and throughput during training as MultiLoRA inherits zero inference overhead from original LoRA. Our benchmark protocol involves training LLaMA-7B on sequences of 1024 tokens using 8 A100 GPUs and recording the peak VRAM usage and throughput[3]. Deepspeed ZeRO-3 and model parameter offload are activated to better evaluate impacts brought by MultiLoRA.




(a) Throughput

(b) VRAM

Figure 3: (a) Throughput and (b) peak VRAM usage benchmarked when training LLaMA-7Bwith sequences of 1024 tokens and batch size of 1. $n \times r$ on horizontal axis indicates total rank of LoRA and MultiLoRA. Thanks to high parallelism of MultiLoRA, training throughput is almost identical to LoRA. VRAM usage scales up linearly with the number of parallel LoRA modules.

Results are listed in Figure 3. On the horizontal axis, $n \times r$ denotes the equal total rank of MultiLoRA ($n$ parallel LoRA of rank $r$) and LoRA (one LoRA of rank $n \times r$).

Training throughput is one of the advantages of using MultiLoRA as other PEFT methods. A limitation with full parameter fine-tuning lies in the fact that cached optimizer states can consume a significant portion of the VRAM. Specifically, when training a 7B model with AdamW[32], cached optimizer states can occupy up to 70% of the available VRAM, and over 48% with SGD[33]. Thanks to the significantly reduced number of trainable parameters, finite VRAM can be leveraged to load more data samples, leading to larger training throughput. Additionally, due to the parallelism inherent in MultiLoRA, multiple LoRA modules do not introduce notable latency and the throughput remains close to

[3]We train MultiLoRA with individual rank of 32.

that of LoRA, around 400 tokens per GPU per second. In our benchmarking, the throughput of MultiLoRA is almost twice that of full parameter fine-tuning (208 tokens per GPU per second).

For VRAM usage, peak memory scales up much faster than LoRA. In order to optimize multiple parallel LoRA modules, multiple copies of activations should be cached in VRAM. Therefore, one major drawback of MultiLoRA is activation VRAM usage scales linearly with number of parallel LoRA modules which can be unaffordable in long sequence training. In our benchmark, training LLaMA-7B with sequences of 1024 tokens with $n = 5$ would use more VRAM than full parameter fine-tuning.

## 5 Understanding MultiLoRA

In this section, we apply SVD on weight update matrices trained with LLaMA-7B in Section 4.1 to investigate why MultiLoRA outperforms LoRA in complex task adaptation. Specifically, subspace similarity and magnitudes of singular value are thoroughly studied for MultiLoRA, LoRA and fine-tuning.

### 5.1 Comparison with Fine-tuning

to demonstrate a higher degree of similarity to full parameter fine-tuning of MultiLoRA, we utilize SVD to compare weight update matrices $\Delta W$ of LoRA and MultiLoRA. Specifically, we focus on comparing the subspace coverage of singular vectors and the magnitudes of singular values.

As for subspace similarity of singular vectors, we follow [6] to use $\phi(\Delta W', \Delta W, i, j)$ in Equation 5, the Frobenius norm of cosine similarity between top-i and top-j singular vectors of two weight update matrices.

$$\phi(\Delta W', \Delta W, i, j) = \frac{\|U_i^\top U_j'\|_F^2}{\min(i, j)} \in [0, 1], \tag{5}$$

where $U_i = U[:, : i]$ and $U_j' = U'[:, : j]$ are stacked top-i and top-j singular vectors.

Moreover, the magnitudes of singular values offer valuable insights into the relative importance of each singular vector. Larger singular values signify a greater contribution to the overall data representation. In Section 3.2, we find $\Delta W^{LoRA}$ is very polarized as proportion of top singular values is largest. By comparing the singular value distribution, we want to find out whether MultiLoRA manages to balance contribution of each singular vectors.

#### 5.1.1 Subspace Comparison

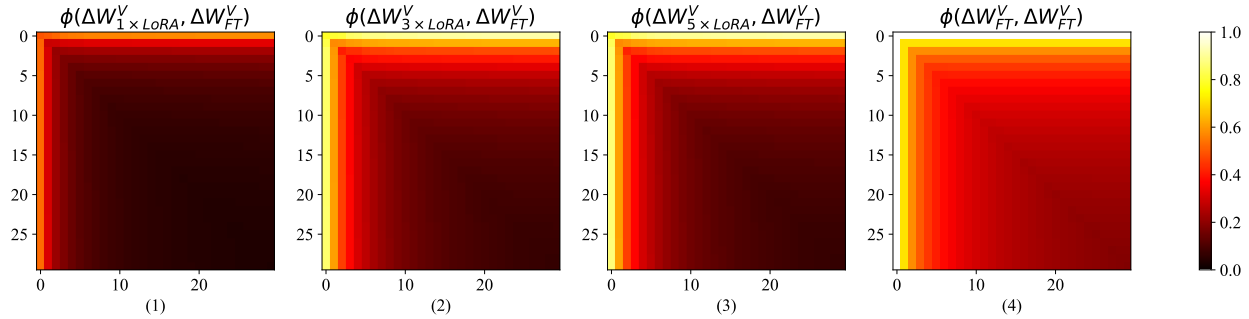



Figure 4: Subspace similarity to fine-tuning of LoRA (**1**), MultiLoRA (**2, 3**) and fine-tuning with a different random seed (**4**). LoRA (**2**) and MultiLoRA (**3**) share same parameter budget but MultiLoRA exhibits stronger subspace similarity to fine-tuning. Heatmap of MultiLoRA$^{n=3}_{r=32}$ does not differ much from that of MultiLoRA$^{n=5}_{r=32}$. Only $i, j \in [1, 30]$ are presented for better visibility.

Orthonormal singular vectors define the "direction" of data transform. By measuring the subspace overlapping with $\phi(\Delta W', \Delta W)$, we can measure the degree of similarity between two transforms. We randomly choose value projection of the $15th$ decoder layer to calculate $\phi(\Delta W^{LoRA}, \Delta W^{FT})$ and $\phi(\Delta W^{MultiLoRA}, \Delta W^{FT})$. Similarity between fine-tuning of two different runs $\phi(\Delta W^{FT'}, \Delta W^{FT})$ is also calculated for reference.

**MultiLoRA resembles fine-tuning more than LoRA in terms of subspace span.** According to visualization in Figure 4, MultiLoRA$^{n=3}_{r=32}$ exhibits stronger resemblance to fine-tuning than LoRA$_{r=96}$ under the same parameter budget,

indicating that subspace of the weight update matrix of MultiLoRA is closer to that of fine-tuning. Heatmap of LoRA is generally dimmer but top singular vectors still present overlapping of subspaces to fine-tuning to some degree.

**Scaling up $n$ does not necessarily augment MultiLoRA subspace similarity to fine-tuning.** Another thing to behold is barely visible difference between $\text{MultiLoRA}_{r=32}^{n=3}$ and $\text{MultiLoRA}_{r=32}^{n=5}$, meaning that increasing parallel LoRA number $n$ does not necessarily make subspace closer to fine-tuning. The same trend can be observed on other weights of different depths in decoder stack (more at Appendix C).

### 5.1.2 Singular Value Distribution Comparison

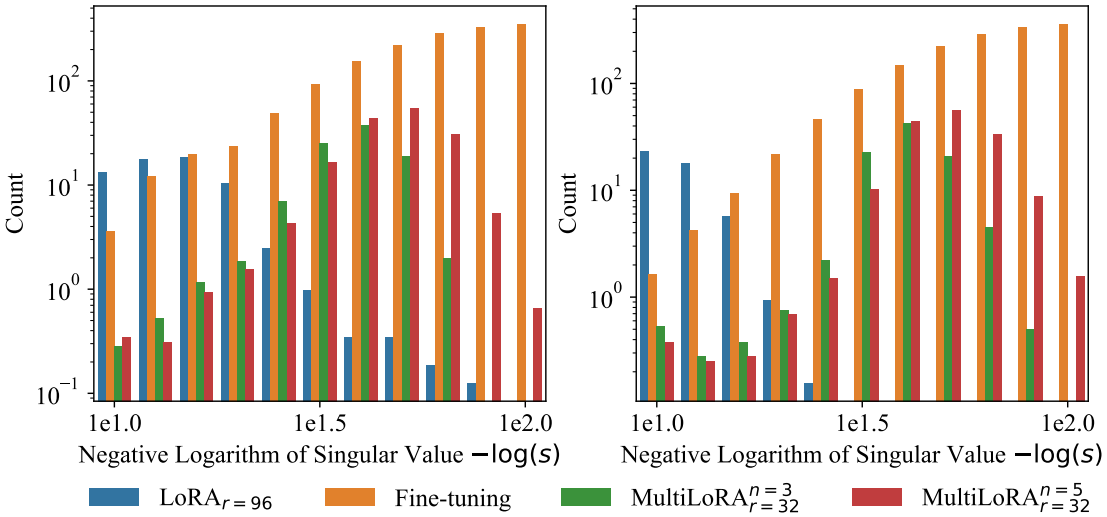



Figure 5: Singular value distribution of weight update matrices $\Delta W$ of *k_proj* (**Left**) and *v_proj* (**Right**). Our proposed MultiLoRA exhibits higher degree of resemblance to fine-tuning. Scaling up $n$ produces more democratic unitary transform contributions.

In previous Section 5.1.1, we measure the subspace similarity between unitary singular vectors but without knowing the importance of aforementioned singular vectors, we cannot conclude the higher resemblance between MultiLoRA and fine-tuning. Thus, we investigate into singular value distribution by plotting histogram of singular value as in Section 3.2.

For $\Sigma = diag(s)$ obtained from SVD$(\Delta W)$, we count the number of $s$ over a series of thresholds and average the statistics of the same module over different depths of decoder layers. We calculate negative logarithms $-\log(s)$ for better visibility since more than 95% of singular values are within $[0, 1]$. Results are shown in Figure 5.

**MultiLoRA balances subspace contributions compared to LoRA.** MultiLoRA shows similar distribution as fine-tuning where number of singular value of MultiLoRA decreases with its magnitude. Given the explicit low rank $r << d$, LoRA shows heavy reliance on a small group of top singular vectors but MultiLoRAdemocratizes contributions of singular vectors.

**Scaling up $n$ makes MultiLoRA amplify features at a more fine-grained level.** Comparing $\text{MultiLoRA}_{r=32}^{n=3}$ and $\text{MultiLoRA}_{r=32}^{n=5}$, histograms of $\{s| -\log(s) > 1e1.6\}$ are almost identical but $\text{MultiLoRA}_{r=32}^{n=5}$ shows wider spectrum as proportion of small singular values increases. A wider spectrum covering small singular values enables more fine-grained fitting of $\Delta W$ as fine-tuning.

## 5.2 Comparison among MultiLoRA

In this section, to demonstrate MultiLoRA accomplishes the design goal of depolarization, we compare in pair sub-LoRAs. From heatmap, subspace similarity between top-1 singular vectors is around 0.6. Comparison between $\Delta W_{i=5}$ and $\Delta W_{i=3}$ shows relatively low similarity. The variance of subspace similarities indicates a more fine-grained pattern decomposition.

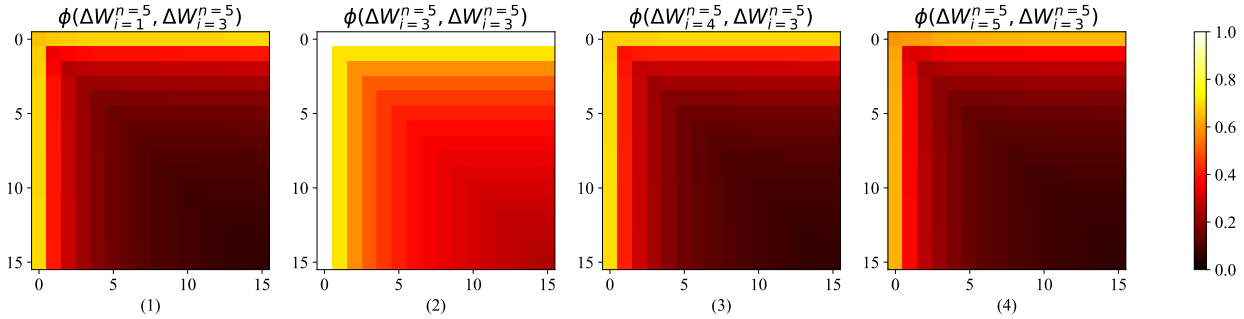



Figure 6: Subspace similarity between parallel module of MultiLoRA. We analyze the MultiLoRA targeting *down_proj* in the first decoder layer. Each individual module produces close but not identical subspaces, thus augmenting the general expressiveness of MultiLoRA.

### 5.3 Underlying Mechanisms of LoRA and MultiLoRA

In previous sections, we study the difference in subspace similarity and singular value distribution by applying SVD on weight update matrices of experimented methods. Our observations shed light on underlying mechanisms of LoRA and MultiLoRA. From the singular value distribution of fine-tuning, we learn that fine-tuning fits residual weights by aggregating a large number (usually equals to rank of weight matrix) of relatively less important unitary transforms. Given the low rank limitation, LoRA and MultiLoRA fits residual weights with $r \ll min(d, k)$ unitary transforms. The low subspace similarity to fine-tuning and dominance in singular value distribution observed in LoRA show that LoRA tends to decompose the residual weights into unitary transforms of large importance. Meanwhile, MultiLoRA democratizes influences of unitary transforms by assigning smaller importance to its unitary transforms similar to fine-tuning. With democratized unitary subspaces, MultiLoRA produces better complex multi-task learning performance.

## 6 Conclusion

In conclusion, our study introduces MultiLoRA, a novel approach that enhances multi-task adaptation in language models. By mitigating the dominance of unitary transforms of LoRA, we successfully improve performance in complex multi-task scenarios. Our proposed method focuses on scaling LoRA modules horizontally and modifying parameter initialization to reduce parameter dependency, thereby creating more balanced unitary subspaces. Additionally, we construct a comprehensive dataset covering a wide range of tasks of interest for generative LLMs. Through extensive experimentation, we have demonstrated that MultiLoRA outperforms single LoRA and achieves comparable performance to fine-tuning across multiple benchmarks and model scales. MultiLoRA stabilizes multi-task adaptation especially for smaller models. Furthermore, our investigation into weight update matrices reveals a significant reduction in dependency on top singular vectors and a more equitable contribution of unitary subspaces in MultiLoRA. Overall, MultiLoRA provides an efficient and effective solution for multi-task adaptation in language models.

## References


[1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, pages 4171–4186. Association for Computational Linguistics, 2019.

[2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.

[3] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized BERT pretraining approach. *CoRR*, abs/1907.11692, 2019.

[4] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models. *ArXiv*, abs/2302.13971, 2023.

[5] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models. *Trans. Mach. Learn. Res.*, 2022, 2022.

[6] Edward Hu, Yelong Shen, Phil Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.

[7] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. *CoRR*, abs/1902.00751, 2019.

[8] Xiao Liu, Kaixuan Ji, Yicheng Fu, Zhengxiao Du, Zhilin Yang, and Jie Tang. P-tuning v2: Prompt tuning can be comparable to fine-tuning universally across scales and tasks. *CoRR*, abs/2110.07602, 2021.

[9] Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin Raffel. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. In *NeurIPS*, 2022.

[10] Rabeeh Karimi Mahabadi, Sebastian Ruder, Mostafa Dehghani, and James Henderson. Parameter-efficient multi-task fine-tuning for transformers via shared hypernetworks. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL/IJCNLP 2021, (Volume 1: Long Papers), Virtual Event, August 1-6, 2021*, pages 565–576. Association for Computational Linguistics, 2021.

[11] Zhen Wang, Rameswar Panda, Leonid Karlinsky, Rogério Feris, Huan Sun, and Yoon Kim. Multitask prompt tuning enables parameter-efficient transfer learning. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.

[12] Deze Wang, Boxing Chen, Shanshan Li, Wei Luo, Shaoliang Peng, Wei Dong, and Xiangke Liao. One adapter for all programming languages? adapter tuning for code search and summarization. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*, pages 5–16. IEEE, 2023.

[13] Yaqing Wang, Sahaj Agarwal, Subhabrata Mukherjee, Xiaodong Liu, Jing Gao, Ahmed Hassan Awadallah, and Jianfeng Gao. Adamix: Mixture-of-adaptations for parameter-efficient model tuning. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, pages 5744–5760. Association for Computational Linguistics, 2022.

[14] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. Deepspeed- inference: Enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis, Dallas, TX, USA, November 13-18, 2022*, pages 46:1–46:15. IEEE, 2022.

[15] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model. `https://github.com/tatsu-lab/stanford_alpaca`, 2023.

[16] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021.

[17] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *CoRR*, abs/2110.14168, 2021.

[18] Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. Superglue: A stickier benchmark for general-purpose language understanding systems. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pages 3261–3275, 2019.

[19] Ning Ding, Yujia Qin, Guang Yang, Fuchao Wei, Zonghan Yang, Yusheng Su, Shengding Hu, Yulin Chen, Chi-Min Chan, Weize Chen, et al. Delta tuning: A comprehensive study of parameter efficient methods for pre-trained language models. *arXiv preprint arXiv:2203.06904*, 2022.

[20] Ruidan He, Linlin Liu, Hai Ye, Qingyu Tan, Bosheng Ding, Liying Cheng, Jia-Wei Low, Lidong Bing, and Luo Si. On the effectiveness of adapter-based tuning for pretrained language model adaptation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL/IJCNLP 2021, (Volume 1: Long Papers), Virtual Event, August 1-6, 2021*, pages 2208–2222. Association for Computational Linguistics, 2021.

[21] Elad Ben Zaken, Yoav Goldberg, and Shauli Ravfogel. Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*, pages 1–9. Association for Computational Linguistics, 2022.

[22] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *NeurIPS*, 2022.

[23] Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. Adaptive budget allocation for parameter-efficient fine-tuning. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.

[24] Yuning Mao, Lambert Mathias, Rui Hou, Amjad Almahairi, Hao Ma, Jiawei Han, Scott Yih, and Madian Khabsa. Unipelt: A unified framework for parameter-efficient language model tuning. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*, pages 6253–6264. Association for Computational Linguistics, 2022.

[25] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *2015 IEEE International Conference on Computer Vision, ICCV 2015, Santiago, Chile, December 7-13, 2015*, pages 1026–1034. IEEE Computer Society, 2015.

[26] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, AISTATS 2010, Chia Laguna Resort, Sardinia, Italy, May 13-15, 2010*, volume 9 of *JMLR Proceedings*, pages 249–256. JMLR.org, 2010.

[27] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Z. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pages 8024–8035, 2019.

[28] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *CoRR*, abs/2305.14314, 2023.

[29] Sadhika Malladi, Tianyu Gao, Eshaan Nichani, Alex Damian, Jason D. Lee, Danqi Chen, and Sanjeev Arora. Fine-tuning language models with just forward passes. *CoRR*, abs/2305.17333, 2023.

[30] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. Peft: State-of-the-art parameter-efficient fine-tuning methods. `https://github.com/huggingface/peft`, 2022.

[31] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2020, Virtual Event / Atlanta, Georgia, USA, November 9-19, 2020*, page 20. IEEE/ACM, 2020.

[32] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019.

[33] Sebastian Ruder. An overview of gradient descent optimization algorithms. *CoRR*, abs/1609.04747, 2016.

[34] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, October 2020. Association for Computational Linguistics.

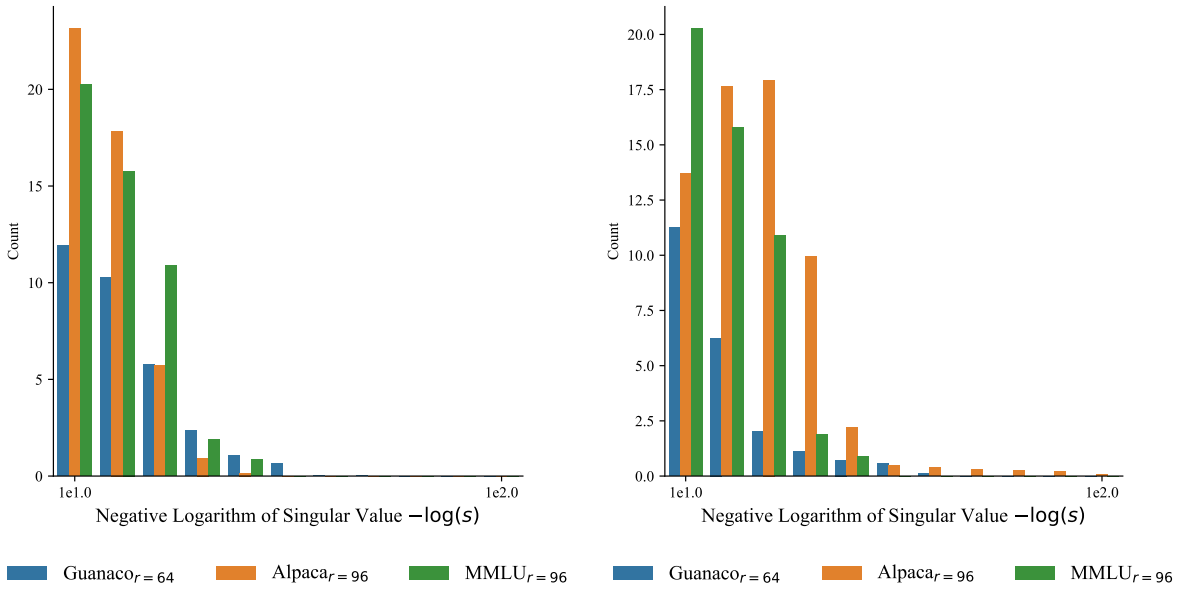
# A Hyperparameters

We list hyperparameters used in our experiments in the Table 3. Batch size of 32 is achieved for LLaMA-30B and LLaMA-65B with gradient accumulation. We use default values give by Huggingface transformers[34] trainer for most of the optimizer hyperparameters.

| Expeiment | Hyperparameters | Values |
|---|---|---|
| | Batch Size per GPU | 32 |
| | Number of Epochs | 2 |
| Fine-Tune | Learning Rate | 5e-6 |
| | LR Schedule | Linear |
| | Optimizer | AdamW |
| | Warmup Ratio | 0.05 |
| LoRA | Learning Rate | 5e-5 |
| | LR Schedule | Linear |
| | Optimizer | AdamW |
| | Warmup Ratio | 0.05 |
| MultiLoRA | Learning Rate | 5e-5 |
| | LR Schedule | Linear |
| | Optimizer | AdamW |
| | Warmup Ratio | 0.05 |

Table 3: Training Hyperparameters used in our experiments.

# B Singular Value Distribution




(a) *v_proj* (b) *q_proj*

Figure 7: Singular Value Distribution of (a) *v_proj* and (b) *q_proj* of weight update matrices trained on different datasets.

In Section 3.2, we find that LoRA's weight update matrices of are dominated by small group of unitary transforms. To further support this, we analyzed LoRA modules obtained from training on various datasets or using publicly available resources. We use LoRA modules obtained by training on public available datasets (MMLU and Alpaca) or downloading publicly available resources (Guanaco[4]).

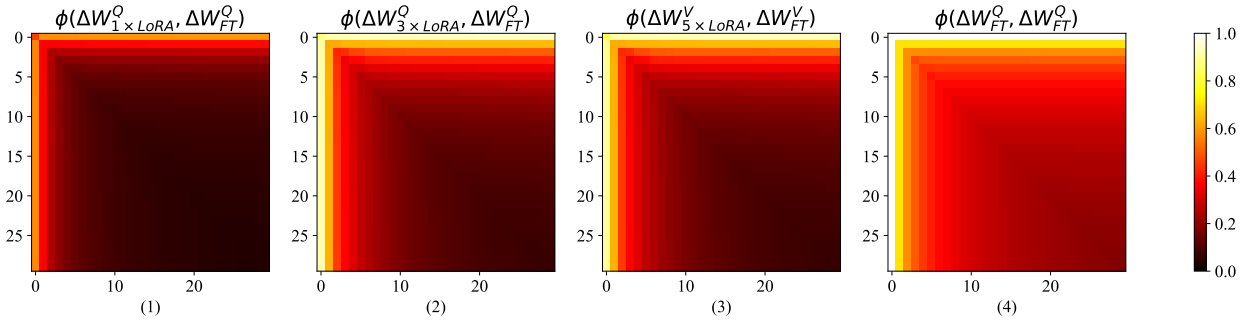
[4]Downloadable at https://huggingface.co/timdettmers/guanaco-7b/tree/main

Figure 7 plots histograms of singular values of weight update matrices of *q_proj* and *v_proj*. To enhance visualization, the negative logarithm of the singular values $(-\log(s))$ is calculated, given that most values are smaller than 0.1

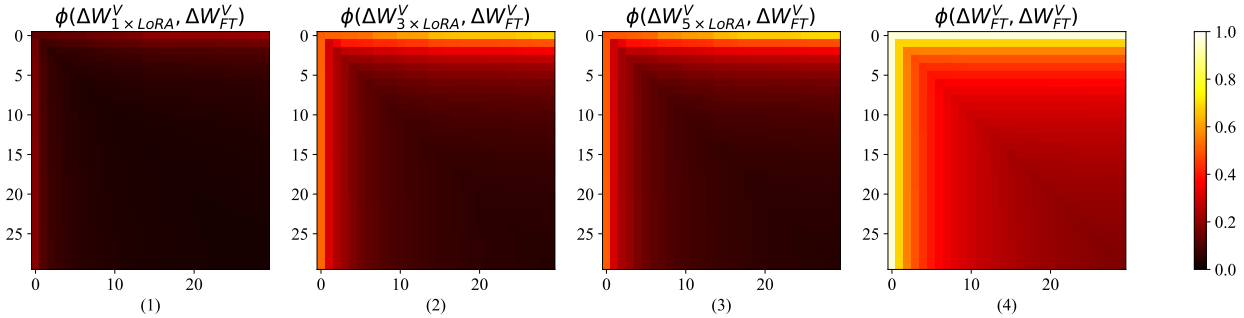
Mean values are used to aggregate statistics across all decoder layers. The histograms for both modules exhibit a striking similarity. The triangular shape of the histograms indicates the dominance of the top singular vectors, as mentioned in Section 3.2. It is worth noting that this dominance arises from the inherent design of LoRA, as we do not deliberately alter its structure or use unconventional datasets.

## C Subspace Similarities of other modules of different depth

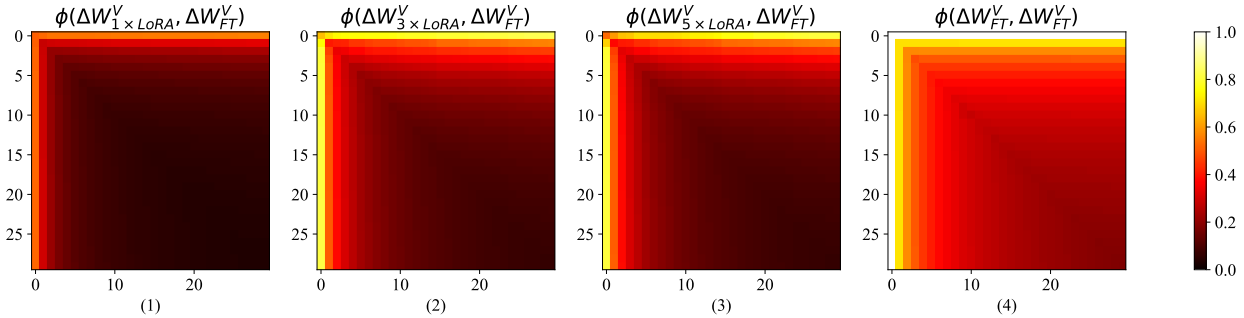
In Section 5.1.1, we use cosine similarity between top singular vectors to measure subspace overlap of $\Delta W$. Here, we present more visualizations on different modules from different depths of the decoder stack.




(a) *up_proj* of MLP in the 28*th* decoder layer.




(b) *q_proj* of self attention in the 1*st* decoder layer.




(c) *k_proj* of self attention in the 14*th* decoder layer.

Figure 8: Subspace similarity of LoRA and MultiLoRA to fine-tuningof different modules at different depths.

We randomly choose *up_proj*, *q_proj* and *k_proj* of MLP and self attention module at different depths to compare weight update matrices of LoRA and MultiLoRA to fine-tuning. The heatmap visualization shows higher degree of similarity of MultiLoRA as observed in Section 5.1.1. Our observation sheds light on the mechanism of LoRAthat the residual weight is decomposed into a small number group of unitary transform of large importance. Important unitary transforms

of small number hinders the model handling complicated multi-task learning. Meanwhile, MultiLoRA manages to fits the residual weight more similar to fine-tuning which gathers a larger number of relatively less important transforms.